

QUESTIONS

10 / 10 completed

Integer Array

INTEGER ARRAY

"Write a C program to demonstrate pointer arithmetic on an integer array. Display element values using both array indexing and pointer notation. Show address difference between consecutive elements.

Sample Input:

Enter number of elements: 4

Enter elements:

10 20 30 40

Sample Output:

Using Array Indexing:

arr[0] = 10

arr[1] = 20

arr[2] = 30

arr[3] = 40

Using Pointer Notation:

*(ptr+0) = 10

*(ptr+1) = 20

*(ptr+2) = 30

*(ptr+3) = 40

Address Difference:

Address difference between arr[0] and arr[1] = 4 bytes"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 int main() {
3     int n;
4     printf("Enter number of elements: ");
5     if (scanf("%d", &n) != 1 || n <= 0) {
6         printf("Invalid number of elements.\n");
7         return 1;
8     }
9     int arr[n];
10    printf("Enter elements:\n");
11    for (int i = 0; i < n; i++) {
12        scanf("%d", &arr[i]);
13    }
14    printf("\nUsing Array Indexing:\n");
15    for (int i = 0; i < n; i++) {
16        printf("arr[%d] = %d\n", i, arr[i]);
17    }
18    printf("\nUsing Pointer Notation:\n");
19    for (int i = 0; i < n; i++) {
20        printf("*(arr + %d) = %d\n", i, *(arr + i));
21    }
```

OUTPUT

QUESTIONS

10 / 10 completed

Factorial

FACTORIAL

"Write a C program to find the factorial of a given number using recursion.

Sample Input: Enter a number- 6

Sample Output: Factorial of 6 is 720

"

PROGRAM EDITOR

C



Run

Save

```
1 #include<stdio.h>
2 int factorial(int n)
3 {
4     if(n==0||n==1)
5         return 1;
6     else
7         return n * factorial(n-1);
8 }
9 int main()
10 {
11     int n;
12     printf("Enter a number: \n");
13     scanf("%d",&n);
14
15     printf("Factorial of %d is %d",n,factorial(n));
16     return 0;
17 }
```

OUTPUT

```
Enter a number:
Factorial of 6 is 720
```

QUESTIONS

10 / 10 completed

Array Rotation

ARRAY ROTATION

"Implement array rotation (left/right) for arrays of size 10 by K positions without using extra space. Use a reversal algorithm. The program must validate K ($0 \leq K < N$), show original -> rotated -> reversal steps visually, compute the sum difference, and support multiple rotations.

Sample Input/Output:

Enter 10 numbers: 1 2 3 4 5 6 7 8 9 10

Rotate Left (L)/Right (R): L

By positions: 3

Original: [1 2 3 4 5 6 7 8 9 10]

Step1: Reverse 0-2 -> [3 2 1 4 5 6 7 8 9 10]

Step2: Reverse 3-9 -> [3 2 1 10 9 8 7 6 5 4]

Step3: Reverse all -> [4 5 6 7 8 9 10 1 2 3]

Sum difference: 0"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #define SIZE 10
5 void printArray(int arr[], int size) {
6     printf("[");
7     for (int i = 0; i < size; i++) {
8         printf("%d", arr[i]);
9         if (i < size - 1) printf(" ");
10    }
11    printf("]\n");
12 }
13 void reverseSubArray(int arr[], int start, int end) {
14     while (start < end) {
15         int temp = arr[start];
16         arr[start] = arr[end];
17         arr[end] = temp;
18         start++;
19         end--;
20     }
21 }
```

OUTPUT

QUESTIONS

10 / 10 completed

Sparse Matrices

SPARSE MATRICES

Represent two sparse matrices (mostly zeros) using arrays of triples [row, col, value]. Read dimensions and non-zero terms for two matrices. Add them, store result in sparse form (sorted by row then col), and display dense matrix form. Max 10x10; ignore zeros.

Sample Input (Matrix 1):

```
3 3 3
0 1 5
1 0 8
2 2 2
```

Sample Input (Matrix 2):

```
3 3 2
0 1 3
2 2 7
```

Sample Output (Sum Dense):

```
0 8 0
8 0 0
0 0 9
```

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #define MAX_TERMS 101
3 struct Term {
4     int row;
5     int col;
6     int value;
7 };
8 void readSparseMatrix(struct Term matrix[], const char* name) {
9     printf("Enter total rows, columns, and number of non-zero elements for %s:\n", name);
10    scanf("%d %d %d", &matrix[0].row, &matrix[0].col, &matrix[0].value);
11    printf("Enter triplets (row, col, value) one by one:\n");
12    for (int i = 1; i <= matrix[0].value; i++) {
13        scanf("%d %d %d", &matrix[i].row, &matrix[i].col, &matrix[i].value);
14    }
15 }
16 void addSparseMatrices(struct Term m1[], struct Term m2[], struct Term result[]) {
17     if (m1[0].row != m2[0].row || m1[0].col != m2[0].col) {
18         printf("Error: Matrix dimensions do not match for addition.\n");
19         result[0].value = 0;
20         return;
21     }
22 }
```

OUTPUT

```
Compilation Error:
C:\Windows\TEMP\sandbox_192511421RD_6a55c932b6e710.26192566\main.c:1:1: error: expected
 '=', '(', '{', 'asm' or '__attribute__' at end of input
1 | etc
  | ^^^
```

QUESTIONS

10 / 10 completed

Fibonacci Series

FIBONACCI SERIES

Write a C program to print the first n terms of the Fibonacci series ($n \leq 20$) without using recursion. Start with 0 and 1.

Sample Input/Output:

Enter number of terms: 8

Fibonacci series: 0 1 1 2 3 5 8 13

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,a=0,b=1,c,i;
5     printf("Enter number of terms:\n");
6     scanf("%d",&n);
7     printf("Fibonacci series:");
8     for(i=1;i<=n;i++)
9     {
10         printf("%d \t",a);
11         c=a+b;
12         a=b;
13         b=c;
14     }
15     return 0;
16 }
```

OUTPUT

```
Enter number of terms:
Fibonacci series:0    1    1    2    3    5    8    13
```

QUESTIONS

10 / 10 completed

Array

ARRAY

Write a C program to find the 5th element in the given array/

Sample Input: a=[10,20,30,40,50,60]

Sample output: 50

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 int main()
3 {
4     int a[5],i;
5     printf("Enter 5 elements: \n");
6     for(int i=0;i<5;i++)
7         scanf("%d",&a[i]);
8     printf("5th element is = %d",a[4]);
9     return 0;
10 }
```

OUTPUT

```
Enter 5 elements:
5th element is = 50
```

QUESTIONS

10 / 10 completed

Fibonacci series ($N \leq 50$)FIBONACCI SERIES ($N \leq 50$)

Develop a C program to print the first N terms of the Fibonacci series ($N \leq 50$) without using recursion. Use two variables to generate the series iteratively and store the series in an array. The program should also calculate and display the sum of the series and identify the largest term. Input validation is mandatory for N ($1 \leq N \leq 50$).

Sample Input/Output: text

Enter number of terms (1-50): 10

Fibonacci Series: 0 1 1 2 3 5 8 13 21 34

Sum of series: 88

Largest term: 34

Enter number of terms (1-50): 60

Invalid Input! Please enter terms between 1 and 50.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 int main() {
3     int n;
4     unsigned long long fib[50];
5     unsigned long long sum = 0;
6     unsigned long long max = 0;
7     printf("Enter number of terms (1-50): ");
8     if (scanf("%d", &n) != 1 || n < 1 || n > 50) {
9         printf("Invalid input! Please enter a number between 1 and 50.\n");
10        return 1;
11    }
12    unsigned long long a = 0;
13    unsigned long long b = 1;
14    for (int i = 0; i < n; i++) {
15        fib[i] = a;
16        sum += a;
17        if (a > max) {
18            max = a;
19        }
20        unsigned long long next = a + b;
21        a = b;
```

OUTPUT

QUESTIONS

10 / 10 completed

Unsorted Array

UNSORTED ARRAY

Write a program to find the missing element for the given unsorted array of sequence numbers.

Sample Input: a=[10,9,7,4,3,1,2]

Sample output: 5,6,8

PROGRAM EDITOR

C

Run

Save

```
1 def find_missing_elements(arr):
2     if not arr:
3         return []
4     num_set = set(arr)
5     min_val = min(arr)
6     max_val = max(arr)
7     missing = [num for num in range(min_val, max_val + 1) if num not in num_set]
8     return missing
9 sample_input = [10, 9, 7, 4, 3, 1, 2]
10 output = find_missing_elements(sample_input)
11 print("Sample output:", ", ".join(map(str, output)))
```

OUTPUT

QUESTIONS

10 / 10 completed

Fib series-iterative & recur

FIB SERIES-ITERATIVE & RECUR

Write a C program to generate Fibonacci series up to N terms using both iterative and recursive methods. The program should display both outputs and compare execution time in milliseconds. Also display total recursive calls made.

Sample Input:

Enter N: 7

Sample Output:

Iterative: 0 1 1 2 3 5 8

Recursive: 0 1 1 2 3 5 8

Recursive Calls: 13

Iterative Time: 0.001 ms

Recursive Time: 0.005 ms

Test Case:

Enter N: 1

Output:

0

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <time.h>
3 long long recursive_call_count = 0;
4 long long fibonacci_recursive(int n) {
5     recursive_call_count++;
6     if (n <= 0) return 0;
7     if (n == 1) return 1;
8     return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2);
9 }
10 void generate_recursive_series(int n) {
11     printf("Recursive Series: ");
12     clock_t start = clock();
13     for (int i = 0; i < n; i++) {
14         printf("%lld ", fibonacci_recursive(i));
15     }
16     clock_t end = clock();
17     printf("\n");
18     double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000.0;
19     printf("Recursive Execution Time: %.4f ms\n", time_taken);
20     printf("Total Recursive Calls Made: %lld\n", recursive_call_count);
21 }
```

OUTPUT

QUESTIONS

10 / 10 completed

Structure

STRUCTURE

Write a C program to store student details (Roll No, Name, Marks) using structure and save them into a file. Display highest mark, class average, and total records stored.

Sample Input:

Enter number of students: 2

101 Ravi 75

102 Kiran 85

Sample Output:

Highest Mark: 85 (Kiran)

Class Average: 80.00

2 records stored successfully.

Test Case:

1 student entry → Average = Marks of that student

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Define the structure for student details
5 struct Student {
6     int roll_no;
7     char name[50];
8     float marks;
9 };
10
11 int main() {
12     int n;
13
14     printf("Enter number of students: ");
15     if (scanf("%d", &n) != 1 || n <= 0) {
16         printf("Invalid number of students.\n");
17         return 1;
18     }
19
20     // Allocate memory for 'n' students
21     struct Student *students = (struct Student *)malloc(n * sizeof(struct Student));
```

OUTPUT